Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

Team 33: SickSense Pathfinder Bi-Weekly Update 1

Jason Agnew, Josiah Faircloth
Samuel Ramos, Paavan Bagla
Sponsor: SICK
TA: Roman Venegas

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

AgriPollinate

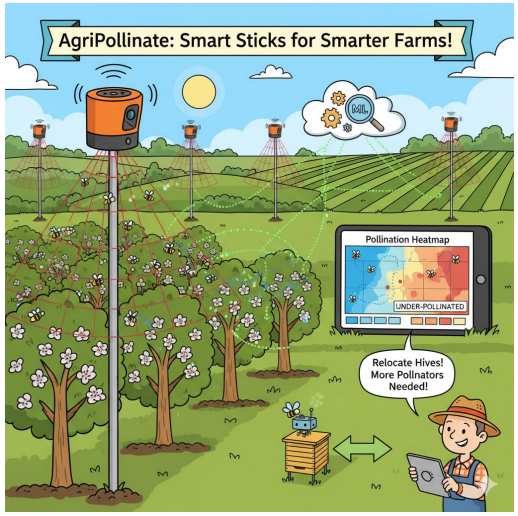
Project Summary

Problem Statement:

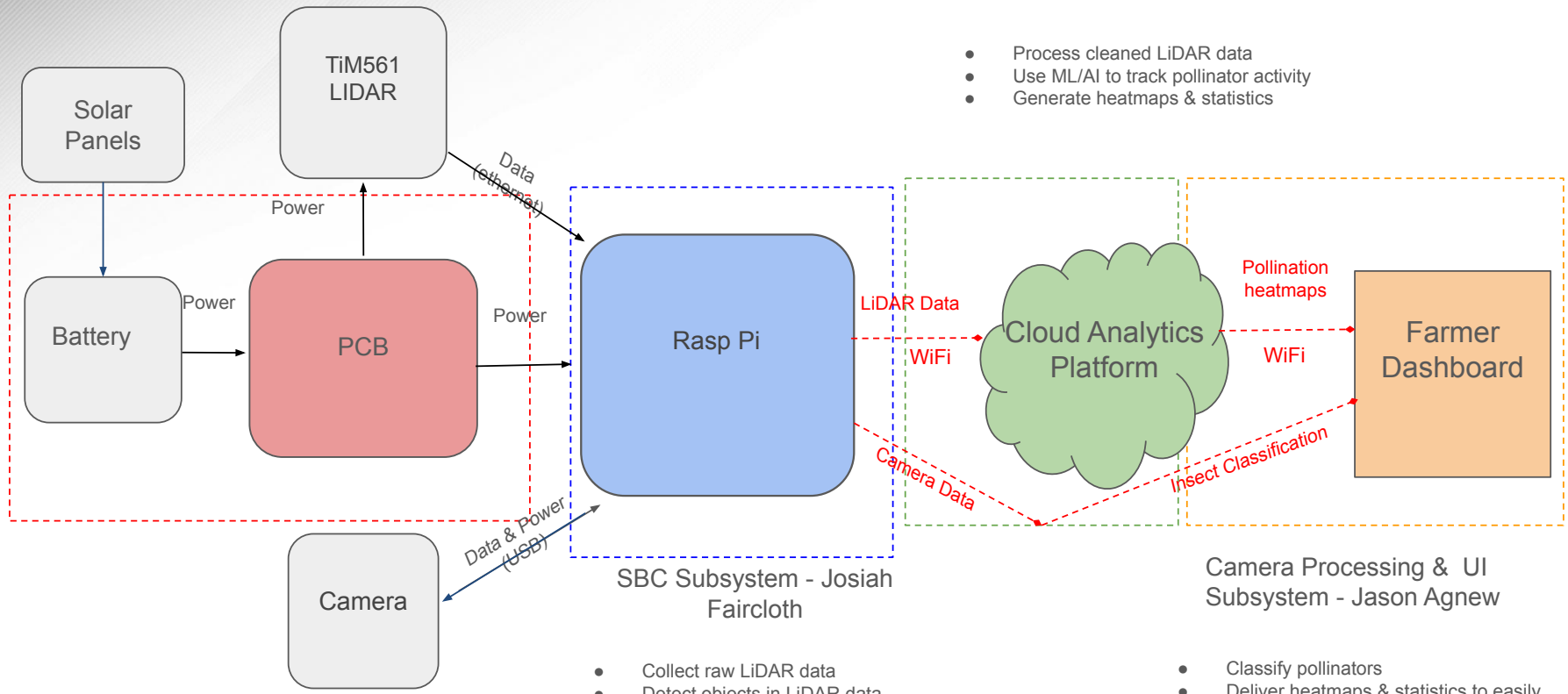
- Farmers lack reliable, real-time insights into pollination activity. Existing methods are reactive, labor-intensive, and often inaccurate, leading to missed pollination deficits that directly reduce crop yield and quality.

Our Solution:

- AgriPollinate is a Light Detection and Ranging (LiDAR)-powered monitoring system that continuously tracks pollinator activity and generates easy-to-read heatmaps.
- Farmers can access these heatmaps through a simple dashboard, allowing them to quickly identify under-pollinated areas, optimize hive placement, and improve crop yield and quality.



Subsystem Overview



LiDAR Processing Subsystem - Paavan Bagla

- Process cleaned LiDAR data
- Use ML/AI to track pollinator activity
- Generate heatmaps & statistics

SBC Subsystem - Josiah Faircloth

- Collect raw LiDAR data
- Detect objects in LiDAR data
- Capture images
- Transmit cleaned data & images

Camera Processing & UI Subsystem - Jason Agnew

- Classify pollinators
- Deliver heatmaps & statistics to easily digestible user interface

Power Subsystem - Samuel Ramos

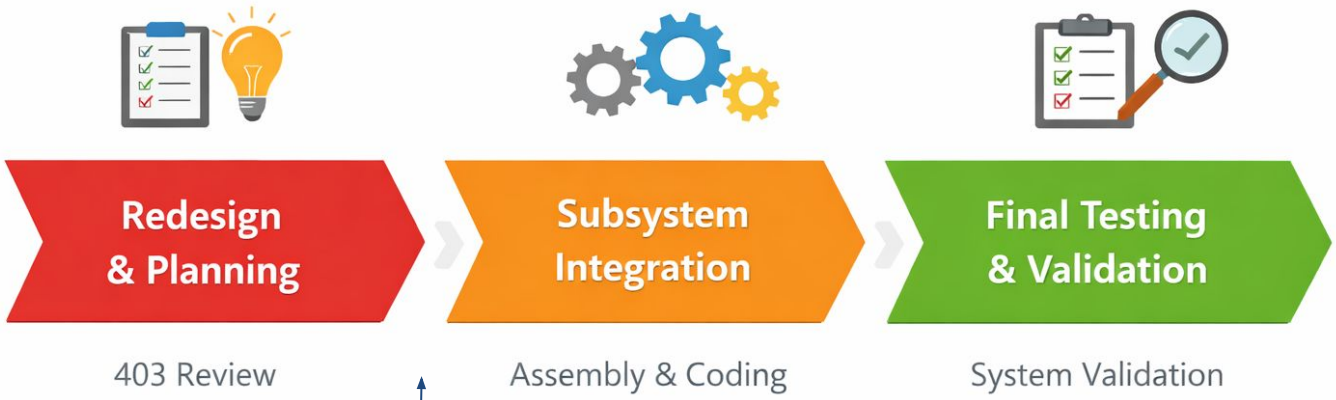
- Supply power to physical components
- Charge Controller for the Solar Panel

Major Project Changes

Major changes based on lessons from 403:

- Pivoted from camera first perception to LiDAR first detection due to reliability and lighting limitations observed in 403, requiring development of a machine learning model based on LiDAR scan data
- Pivoted from browser-based machine learning using TensorFlow.js to a robust Flask API integrated with YOLOv8, significantly enhancing object detection accuracy.

Project Timeline



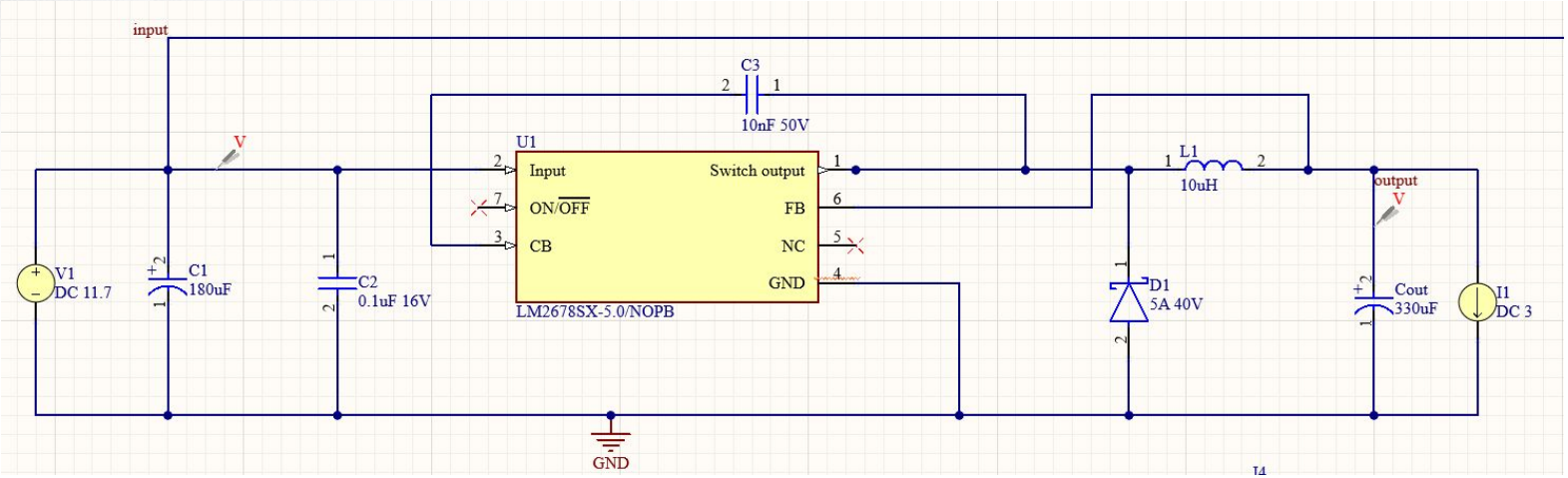
Solar Power Management Circuit

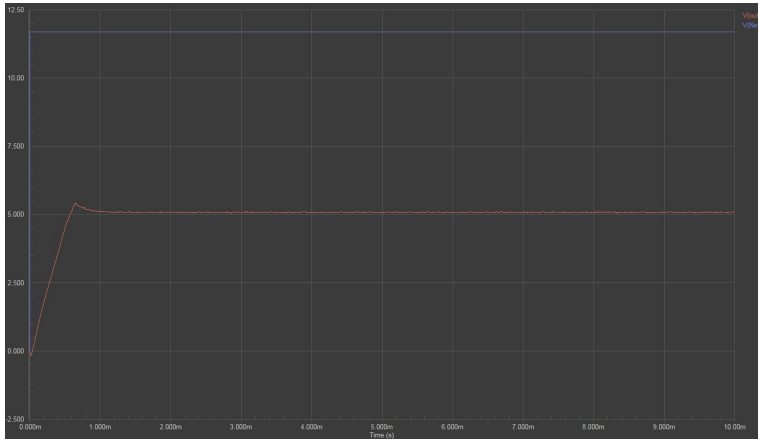
Samuel Ramos

Accomplishments since 403 ~21 hrs of effort	Ongoing progress/problems and plans until the next presentation
- Redesigned PCB board - Revised both schematics with components swaps and additional components	- Simulation of new circuits - Ordering of New Board & Components - Packaging of Project (CAD, 3D printing)

Solar Power Management Circuit

Samuel Ramos



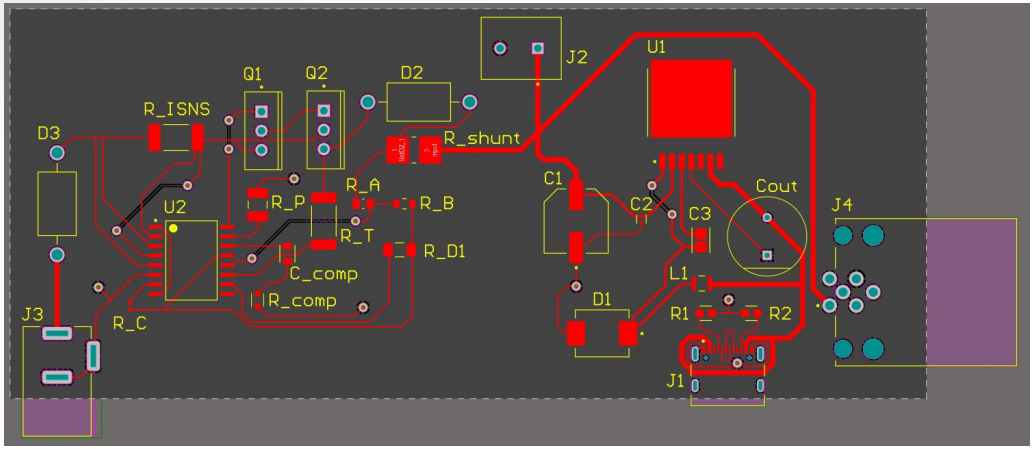


Simulation of Step-converter, showing a mostly flat 5V out under a 3A load.

Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

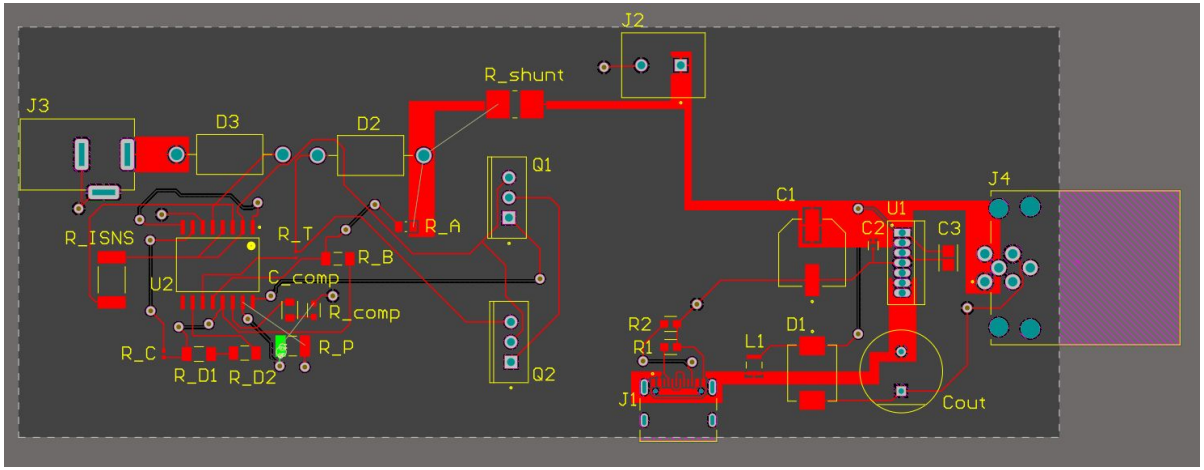
Solar Power Management Circuit

Samuel Ramos



New Design

Old Design



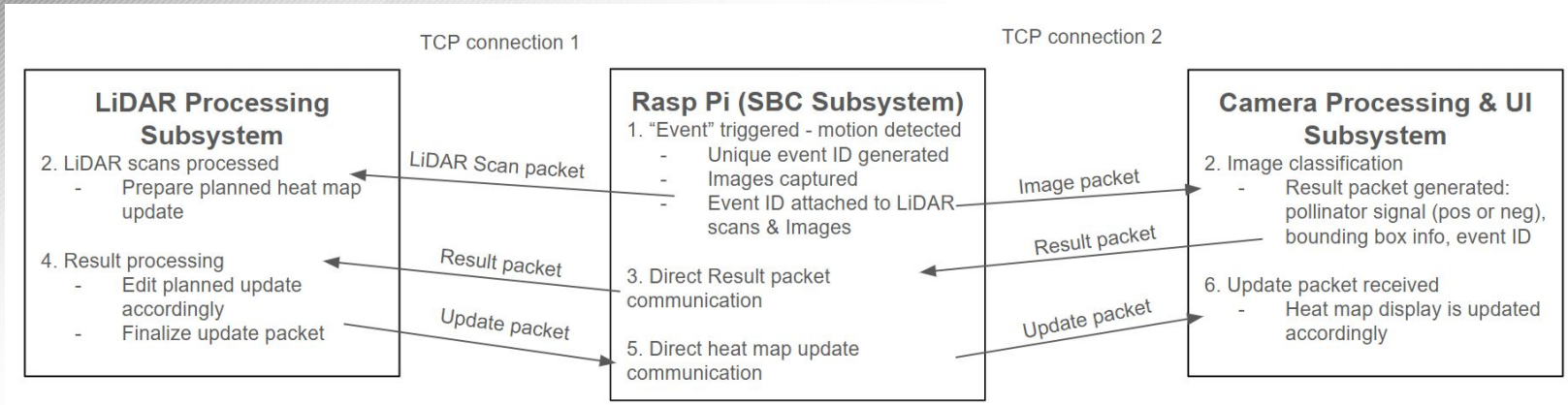
Single Board Computer Subsystem

Josiah Faircloth

Accomplishments since 403 10 hrs of effort	Ongoing progress/problems and plans until the next presentation
Made preparatory program modifications to prepare for integration - Separate TCP connections Designed and Implemented a communication data flow protocol	Testing communication flow & evaluating computational load on Pi Testing & improving algorithm, optimize parameters, and adding new filtering features to improve performance

SBC Subsystem

Josiah Faircloth




```python
class PacketHeader:
    """
    Packet header containing event ID and packet identifier.

    Header format:
    - Event ID: 6 digits before decimal, 2 after (from time.time())
    - Packet Identifier: 4-digit integer
    - Total: variable length JSON for flexibility
    """
```

```python
def to_json(self) -> str:
    """Serialize header to JSON string"""
    return json.dumps({
        "event_id": self.event_id,
        "packet_id": self.packet_id
    })

@staticmethod
def from_json(json_str: str) -> 'PacketHeader':
    """Deserialize header from JSON string"""
    data = json.loads(json_str)
    return PacketHeader(data["event_id"], data["packet_id"])

def __repr__(self):
    return f"Header(event_id={self.event_id}, packet_id={self.packet_id})"
```

LiDAR Processing & Heatmaps Subsystem

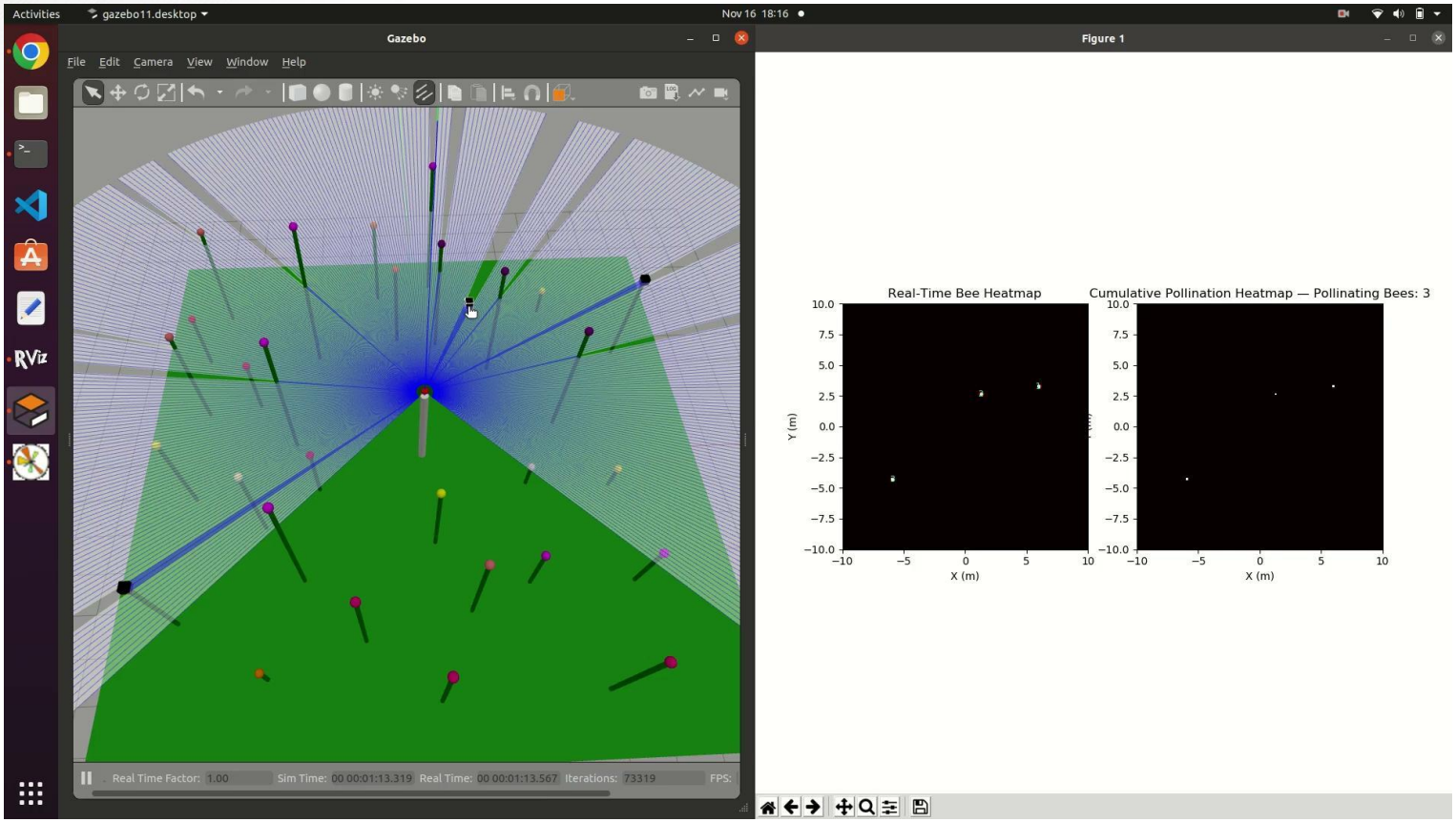
Paavan Bagla

Accomplishments since 403 10 hrs of effort	Ongoing progress/problems and plans until the next presentation
• LiDAR data acquisition and logging pipeline is fully functional from 403 • Defined a pollination focused detection approach using persistent occupancy instead of flying insect detection	• Defining input features for ML model and baseline classifier performance • Visualize and validate sample pollination events from real LiDAR data • Establish reliable and validated communication from the SBC to receive LiDAR event data from the Raspberry Pi

Dwight Look College of
ENGINEERING
TEXAS A&M UNIVERSITY

LiDAR Processing & Heatmaps Subsystem (403)

Paavan Bagla



Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

LiDAR Processing & Heatmaps Subsystem (404)

Paavan Bagla

```python
# Flower configuration
FLOWERS = {
    "A": {
        "angle_indices": [120, 121, 122, 123],
        "background_dist": 0.60
    }
}

DIST_THRESHOLD = 0.08 # If background is 0.60 m, object must be closer than: 0.60 - 0.08 = 0.52 m
START_CONFIRM_SCANS = 3 # object must be present for 3 scans in a row to count as a visit
END_CONFIRM_SCANS = 3 # object must be gone for 3 scans in a row to end the visit
```

```python
def main():
    timestamp = datetime.now().strftime("%m-%d-%Y_%H.%M.%S")
    out_file = f"flower_events_{timestamp}.jsonl"

    flower_state = {
        fid: {
            "active": False, # is a flower currently occupied
            "start_time": None, # time when occupancy started
            "start_count": 0, # consecutive scans confirming occupancy start
            "end_count": 0, # consecutive scans confirming occupancy end
            "distance_series": [] # recorded distances during occupancy
        }
        for fid in FLOWERS
    }
```

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

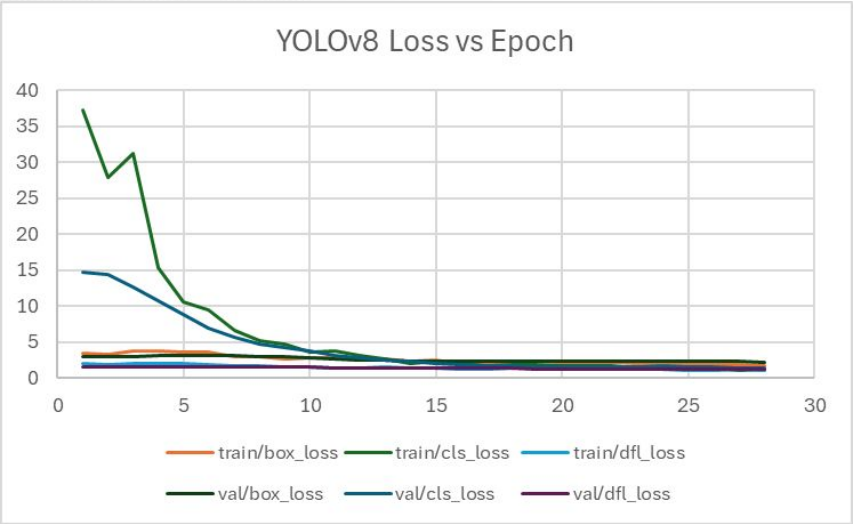
Camera Processing & UI Subsystem

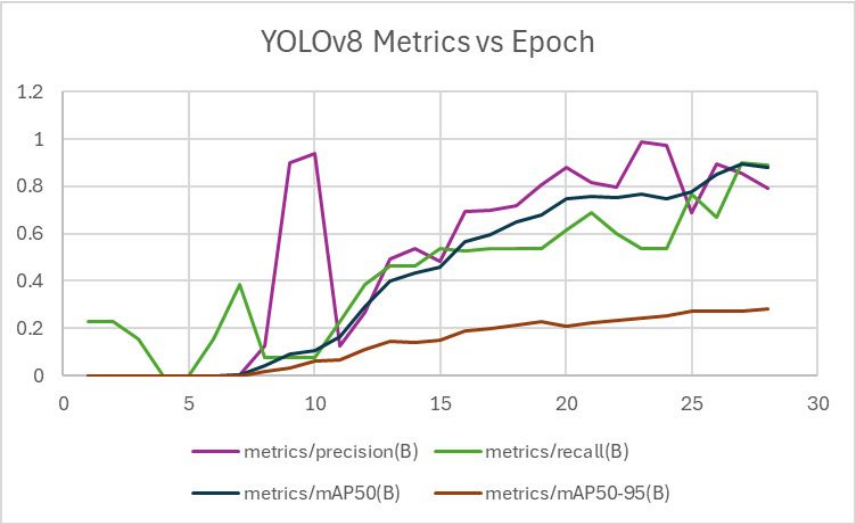
Jason Agnew

Accomplishments since 403 15 hrs of effort	Ongoing progress/problems and plans until the next presentation
• Completed annotation of real field images in Label Studio; exported bounding box data in YOLO format ready for training. • Set up proper conda environment on the HPRC to support YOLOv8 GPU training • Developed and trained an initial YOLOv8 object detection model focused on identifying bees using the custom annotated dataset.	• Continue gathering additional field images, especially including other insect species, to expand training data and improve model accuracy and robustness. • Establish a robust interface for sending insect detection results (species classification and bounding box coordinates) back to the Raspberry Pi, using the TCP protocol. • Prepare workflow for incremental inclusion of newly collected insect species as additional object classes in YOLO annotations and training. • Add more features to the frontend.

Camera Processing & UI Subsystem

Jason Agnew





- box_loss: Are the bounding boxes in the right place?
- cls_loss: Prediction of correct class
- dfl_loss: Are the box edges localized sharply?

- Precision: When I say I found a bee, am I right?
- Recall: Of all possible bees, how many did I find?
- mAP50: How good is the average detection (at 50% IoU)?
- mAP50-95: How well-detailed are my boxes, across multiple levels of overlap?

Execution Plan

Execution Plan

Assigned	Task	Date	Status
Paavan	Define LiDAR ML sample structure and SBC communication requirements for pollination event classification	1/27/2026	Completed
Paavan	Implement flower occupancy event extraction and verify event duration and stability metrics	2/2/2026	Work in Progress
Paavan	Complete reliable LiDAR event communication from Raspberry Pi to SBC as specified in subsystem interface definition	2/4/2026	Planned
Samuel	Redesigned PCB with minimal swap in components	1/22/2026	Completed
Samuel	Simulation of new schematics which contain more changes in components	2/1/2026	Work in Progress
Samuel	Designing the packaging for our project using CAD software	2/8/2026	Planned
Samuel	Soldering of new PCB	2/16/2026	Planned
Samuel	Testing of new PCB under various conditions	2/23/2026	Planned
Josiah	Communication Management Between Subsystems	1/27/2026	Completed
Josiah	Algorithm Improvement & Data Reduction	2/5/2026	Work in Progress
Josiah	Memory Management & Data Retention	2/19/2026	Planned
Josiah	Automatic Network Handling	3/12/2026	Planned
Josiah	Edge Case & Fault Tolerance Programming	3/26/2026	Planned
Josiah	Full System Integration	4/9/2026	Planned
Jason	et up YOLOv8 training/testing environment on HPRC (conda, pip, etc	1/22/2026	Completed
Jason	Train first YOLOv8 "bees only" detection model	1/22/2026	Completed
Jason	Evaluate model on validation images	1/29/2026	Work in Progress
Jason	Build initial Python API/inference script (Flask) for local testing	1/29/2026	Work in Progress
Jason	ıtinue collecting/annotating field data (including possible backgroun	2/5/2026	Planned
Jason	)LO model (tune hyperparameters, do new training runs as annotatic	2/5/2026	Planned
Jason	ɪelop the TCP protocol implementation (send detection to Raspberry	2/5/2026	Planned
Jason	Integrate backend Python API and Raspberry Pi TCP communication	2/5/2026	Planned

Dwight Look College of ENGINEERING TEXAS A&M UNIVERSITY

Validation Plan

Validation Plan

Paragraph #	Test Name	Success Criteria	Methodology	Status	Responsible Engineer(s)
3.2.3.3	LiDAR Scan Acquisition and Stability	Continuous LiDAR scans received and logged with no dropped frames for ≥ 10 minutes	Run LiDAR on Raspberry Pi and log scan timestamps to verify stable scan rate	Completed	Paavan
3.2.1.5	Pollination Event Extraction Validation	Persistent occupancy events detected at known flower locations for ≥ N consecutive scans	Place small object at calibrated flower positions and verify event start, duration, and end logic	Work in Progress	Paavan
3.2.1.10	LiDAR ML Sample Integrity Test	Generated ML samples contain correct angle ranges, distance series, and event durations	Log extracted events and manually inspect sample structure and timing consistency	Planned	Paavan
3.2.1.18	Power Efficiency	The system shall operate for a minimum of 6 hours on a single battery chargeor through solar-assisted power.	Continusouly have a load on the PCB which will run for 6 hours	Planned	Samuel
3.2.1.19	Power Delivery	The system shall deliver the correct amount of power to the LiDAR and the Raspberry Pi continuously for 6 hours	While running the load, consisitently check the voltage and current rates at timed intervals	Planned	Samuel
3.2.1.20	Charging Delivery	The solar panel should be able to charge the battery used in the system with at most 2.1 amps	Measuer shunt resistor while battery is in 3 states, depleted, no load, and with load	Planned	Samuel
3.2.1.16	Communication latency	At least 99% of messages are delivered correctly with end-to-end latency under 100 ms and no ordering errors.	Perform sustained bidirectional message exchanges under normal and high-load conditions while logging delivery rates, latency, and message order.	Work in Progress	Josiah
3.2.1.28	Memory efficiency	Disk usage remains below 85% while all data newer than the defined retention period is preserved without corruption	Run continuous data collection until storage limits are approached and verify automated deletion behavior through file timestamps and disk usage logs	Planned	Josiah
3.2.1.4	Detection performance	False positives account for no more than 20–30% of detection triggers while bee detection recall remains at or above 90%.	Replay labeled LiDAR datasets through the updated algorithm and compare detection results against ground-truth annotations	Planned	Josiah
3.2.1.29	Network handling	The system automatically reconnects within 5 seconds of a network interruption without crashing or losing more than one message.	Intentionally disrupt TCP connections during operation and monitor reconnection time, stability, and message continuity.	Planned	Josiah
3.2.1.14	Fault tolerance	The system enters a safe degraded state during faults and fully recovers within 30 seconds without data corruption.	Inject simulated low-visibility inputs, disconnect subsystems, and perform abrupt power cycles while observing recovery behavior and data integrity	Planned	Josiah
3.2.1.30	Full integration	The fully integrated system runs continuously for at least one hour with stable connections and resource usage below defined limits	Operate the LiDAR and camera processing systems simultaneously in a live environment while monitoring performance metrics and system logs	Planned	Josiah
3.2.1.17	YOLOv8 Bee Detection Accuracy	Model achieves ≥ 90% mAP@0.5 on validation set of bee images	Run YOLOv8 on labeled validation images, calculate mAP@0.5 using Ultralytics metrics	Work in Progress	Jason
3.2.1.18	False Positive Rate on Backgrounds	≤ 1 false positive per 50 field/background (no bee) images	Evaluate model on negative images, log detection count; pass if rate is below threshold	Work in Progress	Jason
3.2.1.19	TCP Output Transmission	≥ 95% of detection outputs are successfully transmitted to Raspberry Pi with correct format	Simulate field workflow, send output via TCP, check logs on RPi	Planned	Jason
3.2.1.20	Multi-Bee Detection Robustness	In images with ≥ 2 bees, all bees are detected, boxes overlap with ground truth by ≥ 0.5 IoU	Test model on annotated multi-bee images, manually or script verify IOU for all boxes	Planned	Jason
3.2.1.21	End-to-End Data Flow	Frontend displays detection results (bounding boxes + species) within 2 seconds of image upload	Demo: upload image via web app, timed observation of bounding box/species overlay	Planned	Jason
3.2.1.22	New Species Expansion	Model retrain accepts new annotated classes, adds them to predictions, and maintains at least 85% accuracy for bee class	Add new species, retrain, check detection/classification results for all classes	Planned	Jason

Final slide – thank your audience